P3299R2
Proposal to extend `std::simd` with range constructors

Published Proposal, 2024-10-16

This version:
https://isocpp.org/files/papers/P2876R2.html

Authors:
Daniel Towner (Intel)
Matthias Kretz (GSI)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to extend `std::simd` with range constructors to address issues raised in LEWG and documented in [P3024R0].

Table of Contents

§ 1. Revision History

R1 => R2

- Added poll from St Louis '24 meeting.
- Added wording
- Added note on `std::ranges_to`.
- Added numerous extra notes, clarifications and examples.

R0 => R1

- Added description of erroneous behaviour
- Added examples

§ 2. Motivation

The [P1928R12] proposal outlines a number of constructors and accessors which can be used to move blocks of memory data in and out of a `simd` object, and these are supplemented in [P2664R6] with functions for permuting data to and from memory. However, these functions are potentially unsafe since they rely on representing memory regions using only an iterator to the start of the memory region, and relying on the `simd` objects own size to determine the extent. In [P3024R0] it was suggested that `simd` should include memory safe operations and LEWG asked for an exploration of what such features might look like, what their cost would be, and what trade-offs might have to be made. This paper discusses that exploration.

This paper primarily discusses moving data from memory into a `simd` using a constructor which accepts a range. Exploration of this problem covers all of the main issues that might arise in safe interactions between memory and a `simd` value. Once a direction has been set we will extend this paper to cover the movement of data from a `simd` back into memory.

§ 3. Existing functions for memory accesses

Being able to load the contents of a region of memory into a `simd` is common practice, and essential for many types of algorithm which are written in a data-parallel style. However, [1928R8] may have some issues related to memory safety, and in this section we discuss some of the options to avoid or mitigate these issues.

There are two constructors which allow a memory load, and two related member functions:

```
simd::simd(contiguous_iterator, flags)

simd::simd(contiguous_iterator, mask, flags)

simd::copy_from(contiguous_iterator, flags)

simd::copy_from(contiguous_iterator, mask, flags)
```

We will only discuss the unmasked constructor in this proposal since the other forms can be readily derived from it. There is one exception which we note in its own section below.

The issue with these memory load constructors and members is that they lack memory safety. They will attempt to load a block of memory into a `simd` value without checking the legality of reading from that memory region, potentially allowing access to data which it should not be able to use, or generating a hardware exception (e.g,. page fault). In this section we shall discuss possible ways to improve this behaviour, along with the costs in doing so.

The constructors are not the only place in `simd` which needs to take care of memory safety. The `gather_from` and `scatter_to` functions described in [P2664R6] also suffered from similar access issues, but they have been updated to avoid these issues by replacing single iterators with ranges, and using runtime-checks. However, gather and scatter instructions are relatively rare compared to normal loads, and are expensive instructions anyway, so the cost of adding runtime checks to them has negligible cost. In contrast, a plain memory load is a work-horse instruction, frequently used, and much more sensitive to the imposition of the costs of runtime checks, so we have to be more careful in what we propose for them.

§ 4. Generic guidelines for safe memory accesses

There is one glaring problem with the basic memory access as it is currently defined in `simd`, which is that it provides only an iterator to the start of the source:

```
template&lt;class It, class... Flags>
constexpr basic_simd(It first, simd_flags<Flags...> = {});
```

The absence of information about the extent of the memory source makes it impossible for the implementation to (optionally) check its preconditions. Fundamentally, the fix for this is to provide informa-

tion about the extent of the source which could be done in a few different ways, such as:

```
// Use a pair of iterators to mark the limits
constexpr basic_simd(It begin, It end, simd_flags<Flags...> = {});

// Iterator and size
constexpr basic_simd(It begin, size_type count, simd_flags<Flags...> = {});

// Span with fixed size
constexpr basic_simd(span<T, N>, simd_flags<Flags...> = {});

// Range with dynamic size (possibly unbounded)
constexpr basic_simd(Range r, simd_flags<Flags...> = {});
```

Once we have information about the extent of the memory region then we can add either static compile-time checks, or dynamic run-time checks to use that information to determine correctness and safety. We can then act on that information in a number of ways including throwing an exception on error, defaulting to some safe state, or even choosing to ignore the issue entirely when the programmer knows it is safe to do so or prefers speed over safety.

§ 5. Accessing a source of compile-time extent

In the following examples the elements of the destination `simd` will be filled with the contents of a memory source with compile-time extent:

```
std::array<float, 6> ad;
simd<float, 6> simdA(ad);

std::span<int, 8> sd(d.begin(), 8);
simd<int, 8> simdB(sd);
```

This is the ideal scenario; we have the maximum amount of information possible about the data being moved, and since it is known at compile-time we can enforce whatever our desired behaviour is. In our proposal we think that under these conditions all data from the source should be copied into the

destination. There should be no silent loss of data through truncation, nor should there be silent addition of new values. If the programmer desires either of those behaviour they should make that explicit:

```
std::array<float, 6> ad;
simd<float 8> simdA(ad); // No! There is a mismatch in size causing extra c

// Alternative making the resize explicit.
simd<float, 6> simdA(ad);
auto simdA8 = resize<8>(simdA);
```

Our intention would be to allow construction from anything which has compile-time extent including C-array, `std::array`, `std::span`, and so on. We will either spell these out in the wording, or find a way to generically construct from anything which has compile-time constant size found using `std::extent_v`, `std::tuple_size_v`, and so on. Example implementation:

```
template <typename Rg>
  constexpr inline size_t
  static_range_size = [] -> size_t {
    using T = remove_cvref_t<Rg>;
    if constexpr (requires { typename integral_constant<size_t, T::size()>
      return T::size();
    else if constexpr (requires { typename integral_constant<size_t, T::ext
      return T::extent;
    else if constexpr (extent_v<T> > 0)
      return extent_v<T>;
    else if constexpr (requires {
                         typename integral_constant<size_t, tuple_size<T>:
                         typename ranges::range_value_t<Rg>;
                       })
      {
        if constexpr (tuple_size_v<T> >= 1
                      && same_as<tuple_element_t<0, T>,
                                 ranges::range_value_t<Rg>>)
          return tuple_size_v<T>;
      }
    return dynamic_extent;
  }();
```

We propose that the span constructor is explicit. Our basis for this is to note that [P3116R0] recommends that implicit conversions should only be provided if the types are fundamentally the same. A `std::span` and a `std::simd` may share similarities in storage (both contain a set of element

values), but a `std::simd` also allows an extensive range of parallel operations. It isn't clear cut whether the types are fundamentally the same and so we prefer to be conservative and require the constructor to be explicit. This decision can be revisited if compelling cases are found to make this conversion implicit.

Finally, we can provide deduction guides too, since we know what the size of the output `simd` should be:

```
template <contiguous-range-with-static-extent Rg>
basic_simd(Rg&&)
  -> basic_simd<ranges::range_value_t<Rg>,
                deduce-t<ranges::range_value_t<Rg>, static-range-size<Rg>>>

template <contiguous-range-with-static-extent Rg, typename... Flags>
basic_simd(Rg&&, simd_flags<Flags...>)
  -> basic_simd<ranges::range_value_t<Rg>,
                deduce-t<ranges::range_value_t<Rg>, static-range-size<Rg>>>
```

Recall that we cannot define the necessary deduction guides for the `simd` alias template. Thus, users that want to use CTAD need to use `basic_simd`:

```
std::array<float, 4> data;
std::simd v = data; // compiles only if simd<float>::size() happens to be 4
std::basic_simd w = data; // deduces Abi correctly
```

§ 6. Accessing a source of bounded but unknown extent

When the source has an unknown extent there may be a mismatch between the size of the `simd` and the source's size. We could handle this in one of three ways, from least expensive to most expensive:

1. Make this undefined behaviour
2. Make this defined behaviour which is gracefully handled
3. Make this erroneous behaviour
4. Make this an exception

We look at each of these in turn. We also look at which of these should be the default behaviour.

§ 6.1. Undefined behaviour

This option is equivalent to leaving the status quo of [P1928R12] unchanged, and putting the onus entirely on the user to check that the range is valid. If the source range is too small, the load will go beyond the end of the valid data. If the source range is too big, the data will be truncated to the size of the `simd`.

Even if the range is not used explicitly for bounds checking, the extra information could still be used in debug modes by the compiler to help aid correctness checking, or by static analysis tools, or by mechanisms for contract checking.

§ 6.2. Defined behaviour

With this option the behaviour of the `simd` load will always generate valid data and will always ensure that out-of-bounds reads (or writes for stores) cannot occur.

When the `simd` is too small to hold all the data in the source range, the data will be truncated to what can fit in the `simd` itself.

When the `simd` is bigger than the source range then the extra elements are filled with value initialized data. The memory beyond the end of the smaller source range will not be accessed. This particular behaviour is useful for handling loop remainders/epilogues.

The major costs of this option are the extra instructions to implement the runtime check, and the need to do a partial copy from memory into the simd register. The latter of these might be particular expensive if, for example, the target has to do element-by-element copy from memory to satisfy the need to avoid touching the memory entirely. Also, the introduction of a checked load could introduce a branch into critical load routines where we really don't want branches. Even on modern Intel machines which have masked memory operations, the cost of setting up and acting on the mask for every call is still something we'd prefer to avoid.

We could explicitly opt-in to this behaviour by passing in `simd_flag_partial_loadstore` as a flag.

§ 6.3. Erroneous behaviour

With a minor change in implementation, compared to "defined behaviour" above, the implementation could also support EB on out-of-bounds access. The runtime cost of checking the range size and copy-

ing only the valid elements is exactly the same. The only difference is that the default-initialized simd elements must exhibit EB on usage.

The default-initialization feature above is a valid use-case for implementing a remainder step after a loop over a larger range. So we certainly don't want to unconditionally make default-initialized elements EB. Instead, EB is a replacement for UB, where the runtime cost is acceptable compared to the cost of an out-of-bounds access.

§ 6.4. Exception

The final and most expensive option is to require the source range and the `simd` size to be identical. A runtime check is inserted into the load to check the extents match, and an exception thrown when they don't. This is expensive because of the runtime check, but also because the mechanics of exceptions often disrupt optimized code.

We could explicitly opt-in to this behaviour by passing in `simd_exception` as a flag.

§ 6.5. Choosing the default behaviour

We have listed three options for what the behaviour of a load from an unknown range should be. In each case we can define a suitable flag which opts in to that behaviour: `simd_unchecked`, `simd_flag_partial_loadstore`, and `simd_exception`. We could require the user to always specify what behaviour they want, but this will lead to verbose code. Ideally we should define what the default should be, and realistically the choice is one of undefined behaviour, or defined behaviour. Exceptional behaviour is really too expensive for what is considered to be a performance library.

If we chose to use defined behaviour as the default, where the data will be truncated or default initialized depending upon the source size, then we get repeatable, understandable and sane behaviour whatever the range of data we request. The big downside is that every use of the function will incur a run-time cost to check the bounds and to do something appropriate if a mismatch is found. This introduces a potential performance bottleneck directly into one of the most performance critical parts of `simd`. We could accept this default if we found that the compiler is able to keep the cost of the check as low as possible (e.g., if it can statically analyse the code and remove any redundant checks), but our experience so far is that the check is expensive (TBD: Qualify the expense).

If we chose to use undefined behaviour then we avoid the need to insert a run-time check and action in what is a low-level function with high performance requirements. This most closely matches the status quo of `std::simd` as it is currently defined. By providing the extra bounds information we allow the compiler to do static checking of its own to detect issues, or allow it to emit extra information to allow

range checking in hardware, or through debug facilities. We also get the best possible performance from the `simd` library in the common case.

Given `std::simd`'s primary use as a low-level performance library, we recommend that the default should be undefined behaviour, with the user opting into one of the other more expensive options. Higher level abstractions (such as the simd execution policy discussed later) can use the extra knowledge they have about context to improve on safety.

§ 6.6. Examples

All examples use the following alias `V` and are compiled with `-march=skylake` (32 byte wide SIMD). They implement the parallel part of a reduction over a larger `vector`. The function `g` would need to be a call to `std::reduce(simd)` for a complete reduction. Since that would complicate an analysis we leave it at simply calling an anonymous function `g`.

```
using V = std::simd<float>;
```

This first example incorrectly creates a `span` of static extent at the end of the loop, which, in general, goes beyond the size of the vector.

```
void iter_ub1(const std::vector<float>& x)
{
  V acc {};
  for (auto it = x.begin(); it < x.end(); it += V::size)
  {
    std::span<const float, V::size()> chunk(it, V::size());
    acc += V(chunk);
  }
  g(acc);
}
```

```
iter_ub1(std::vector<float, std::allocator<float> > const&):
  mov  rax, QWORD PTR [rdi]
  mov  rdx, QWORD PTR [rdi+8]
  vxorps  xmm0, xmm0, xmm0
  cmp  rax, rdx
  jnb  .L32
.L33:
  vaddps  ymm0, ymm0, YMMWORD PTR [rax]
```

```
  add  rax, 32
  cmp  rax, rdx
  jb  .L33
.L32:
  jmp  void g<...>
```

So, if the user creates invalid `span`s there's nothing we can do in `simd`. As expected. But lets not create `span`s anymore then. Instead we're going to load from an iterator pair, but potentially calling load with a range size less than 8.

```
void iter_ub2(const std::vector<float>& x)
{
  V acc {};
  for (auto it = x.begin(); it < x.end(); it += V::size)
    acc += std::simd_load(it, x.end());
  g(acc);
}
```

```
iter_ub2(std::vector<float, std::allocator<float> > const&):
  mov  rax, QWORD PTR [rdi]
  mov  rdx, QWORD PTR [rdi+8]
  vxorps  xmm0, xmm0, xmm0
  cmp  rax, rdx
  jnb  .L37
.L38:
  vaddps  ymm0, ymm0, YMMWORD PTR [rax]
  add  rax, 32
  cmp  rax, rdx
  jb  .L38
.L37:
  jmp  void g<...>
```

Now a precondition check inside `simd_load` can see the out-of-bounds access. Contracts checking / debug builds / static analysis could easily catch this. But if the default behavior of `simd_load` is UB, then this is still an invalid program. So, then what happens if we change `simd_load` to partial-load-store when the range is too small?

```
void iter_zero(const std::vector<float>& x)
{
  V acc {};
```

```
  for (auto it = x.begin(); it < x.end(); it += V::size)
    acc += std::simd_load(it, x.end(), std::simd_flag_partial_loadstore);
  g(acc);
}
```

```
iter_zero(std::vector<float, std::allocator<float> > const&):
  mov  rax, QWORD PTR [rdi]
  mov  rcx, QWORD PTR [rdi+8]
  vxorps  xmm0, xmm0, xmm0
  cmp  rax, rcx
  jnb  .L68
  vmovaps  ymm2, ymm0
  mov  r10d, 8
.L65:
  mov  rdx, rcx
  sub  rdx, rax
  cmp  rdx, 28
  jle  .L69
  vmovups  ymm1, YMMWORD PTR [rax]
  add  rax, 32
  vaddps  ymm0, ymm0, ymm1
  cmp  rax, rcx
  jb  .L65
.L68:
  jmp  void g<...>
.L69:
  push  r13
  lea  r13, [rsp+16]
  and  rsp, -32
  push  QWORD PTR [r13-8]
  push  rbp
  mov  rbp, rsp
  push  r13
.L43:
  sar  rdx, 2
  mov  rsi, r10
  vmovaps  YMMWORD PTR [rbp-48], ymm2
  lea  r8, [rbp-48]
  sub  rsi, rdx
  mov  rdx, rax
  cmp  esi, 8
  jnb  .L70
```

```
.L45:
  xor  edi, edi
  test  sil, 4
  je  .L48
  mov  edi, DWORD PTR [rdx]
  mov  DWORD PTR [r8], edi
  mov  edi, 4
.L48:
  test  sil, 2
  je  .L49
  movzx  r9d, WORD PTR [rdx+rdi]
  mov  WORD PTR [r8+rdi], r9w
  add  rdi, 2
.L49:
  and  esi, 1
  je  .L50
  movzx  edx, BYTE PTR [rdx+rdi]
  mov  BYTE PTR [r8+rdi], dl
.L50:
  vmovaps  ymm1, YMMWORD PTR [rbp-48]
  add  rax, 32
  vaddps  ymm0, ymm0, ymm1
  cmp  rax, rcx
  jnb  .L71
.L51:
  mov  rdx, rcx
  sub  rdx, rax
  cmp  rdx, 28
  jle  .L43
  vmovups  ymm1, YMMWORD PTR [rax]
  add  rax, 32
  vaddps  ymm0, ymm0, ymm1
  cmp  rax, rcx
  jb  .L51
.L71:
  mov  r13, QWORD PTR [rbp-8]
  leave
  lea  rsp, [r13-16]
  pop  r13
  jmp  void g<...>
.L70:
  mov  r9d, esi
  xor  edx, edx
```

```
  and  r9d, -8
.L46:
  mov  edi, edx
  add  edx, 8
  mov  r8, QWORD PTR [rax+rdi]
  mov  QWORD PTR [rbp-48+rdi], r8
  cmp  edx, r9d
  jb  .L46
  lea  rdi, [rbp-48]
  lea  r8, [rdi+rdx]
  add  rdx, rax
  jmp  .L45
```

The program is correct now. But it's not as efficient as it should be. There's no way around using UB loads inside the loop. We just need to end the loop earlier and then handle the remainder. For the remainder we can make good use of the `simd_flag_partial_loadstore` behavior.

```
void iter_zero_efficient(const std::vector<float>& x)
{
  V acc {};
  auto it = x.begin();
  for (; it + V::size() <= x.end(); it += V::size)
    acc += std::simd_load(it, x.end());
  acc += std::simd_load(it, x.end(), std::simd_flag_partial_loadstore);
  g(acc);
}
```

```
iter_zero_efficient(std::vector<float, std::allocator<float> > const&):
  mov  rcx, QWORD PTR [rdi]
  mov  rdx, QWORD PTR [rdi+8]
  vxorps  xmm0, xmm0, xmm0
  lea  rax, [rcx+32]
  cmp  rdx, rax
  jb  .L109
.L110:
  vaddps  ymm0, ymm0, YMMWORD PTR [rax-32]
  mov  rcx, rax
  add  rax, 32
  cmp  rdx, rax
  jnb  .L110
.L109:
```

```
  sub  rdx, rcx
  cmp  rdx, 28
  jbe  .L111
  vmovups  ymm1, YMMWORD PTR [rcx]
  vaddps  ymm0, ymm0, ymm1
  jmp  void g<...>
.L111:
  push  r13
  sar  rdx, 2
  mov  esi, 8
  vxorps  xmm1, xmm1, xmm1
  sub  rsi, rdx
  mov  rax, rcx
  lea  r13, [rsp+16]
  and  rsp, -32
  push  QWORD PTR [r13-8]
  push  rbp
  mov  rbp, rsp
  push  r13
  lea  rdi, [rbp-48]
  vmovaps  YMMWORD PTR [rbp-48], ymm1
  cmp  esi, 8
  jnb  .L134
.L113:
  xor  edx, edx
  test  sil, 4
  jne  .L135
  test  sil, 2
  jne  .L136
.L117:
  and  esi, 1
  jne  .L137
.L118:
  vmovaps  ymm1, YMMWORD PTR [rbp-48]
  vaddps  ymm0, ymm0, ymm1
  mov  r13, QWORD PTR [rbp-8]
  leave
  lea  rsp, [r13-16]
  pop  r13
  jmp  void g<...>
.L137:
  movzx  eax, BYTE PTR [rax+rdx]
  mov  BYTE PTR [rdi+rdx], al
```

```
  jmp  .L118
.L136:
  movzx  ecx, WORD PTR [rax+rdx]
  mov  WORD PTR [rdi+rdx], cx
  add  rdx, 2
  and  esi, 1
  je  .L118
  jmp  .L137
.L135:
  mov  edx, DWORD PTR [rax]
  mov  DWORD PTR [rdi], edx
  mov  edx, 4
  test  sil, 2
  je  .L117
  jmp  .L136
.L134:
  mov  r8d, esi
  xor  eax, eax
  and  r8d, -8
.L114:
  mov  edx, eax
  add  eax, 8
  mov  rdi, QWORD PTR [rcx+rdx]
  mov  QWORD PTR [rbp-48+rdx], rdi
  cmp  eax, r8d
  jb  .L114
  lea  rdi, [rbp-48]
  add  rdi, rax
  add  rax, rcx
  jmp  .L113
```

This program is both correct and, at least in the loop, as efficient as possible. With hardware support for masked instructions, a high quality implementation of the partial-init load turns into a single instruction.

§ 7. Overloads for different range specifications

Providing constructors which accept a static or a dynamic extent can solve all of the basic needs of `simd`, but it may be convenient to have additional overloads which simplify some of the common

ways in which `simd` values are constructed. `std::span` already provides a variety of different styles of bound specification, including:

```
span( It first, size_type count );

span( It first, End last );

span( std::type_identity_t<element_type> (&arr)[ N ] ) noexcept;

span( std::array<U, N>& arr );

span( R&& range );

span( std::initializer_list<value_type> il ) noexcept;

span( const std::span<U, N>& source ) noexcept;
```

Many of these styles are already handled by `std::simd`, where constructors are available for compile-time extents or dynamic ranges already, and need not be considered for special treatment here. However, there are two common cases which could be added as utility constructors:

```
constexpr basic_simd( It first, size_type count );
constexpr basic_simd( It first, It last );
```

§ 8. Contiguous or non-contiguous sources

In [P1928R12] and [P2664R6] we make a distinction between loads and stores to contiguous memory regions, which we call copy operations (e.g., `copy_to`, `copy_from`) and non-contiguous operations, which we call `gather_from` and `scatter_to`. We have done so to call out the potentially large performance difference between the two and make it obvious to the user what the expectations should be.

With ranges or spans as the source of data for `simd` constructors we should take similar care to avoid accidental performance traps. We recommend that the new constructors described in this paper should work only on contiguous ranges. If the user wishes to access data in a non-contiguous range then they should explicitly build a suitable contiguous data source, using their expert knowledge to decide how best to achieve that.

§ 9. Range operations for `simd_mask`

In the current definition of `simd_mask` in [P1928R12] there are constructors, and `copy_to`/`copy_from` functions which work with iterators to containers of `bool` values.

As noted in [P2876R1], a container of `bool` values is not an ideal way to store mask-like values. A better alternative for storing mask element bits outside a `simd_mask` is either as a `std::bitset` or as densely packed bits in unsigned integers. [P2876R1] proposes that `simd` provide implicit constructors and accessors for these cases, giving a more efficient and safer mechanism for storing mask bits than the current functions in [P1928R12]. We suggest that the `simd_mask` iterator constructors, and the `simd_mask::copy_from/copy_to` functions are removed entirely, and that `load_from` and `store_to` should only operate on `simd` values.

§ 10. Constructor, member function or free function

At this point we have two constructors loading from a contiguous range: In the static extent case the constructor requires an exact match of `simd` width and range size; in the dynamic extent case the range can be of any size. This mismatch prompts the question whether we're doing something wrong. Why does this compile:

```
std::vector data = {1, 2, 3, 4, 5};
std::simd<int, 4> v(data);
```

but this doesn't compile:

```
std::array data = {1, 2, 3, 4, 5};
std::simd<int, 4> v(data);
```

and this doesn't compile:

```
std::vector data = {1, 2, 3, 4, 5};
std::span<const int, 5> chunk(data, 5);
std::simd<int, 4> v(chunk);
```

We need to resolve this inconsistency. To this end, note that we can characterize these two constructors as "conversion from array" and "load from range" constructors. Under the hood they are both just a `memcpy`, but the semantics we are defining for the `simd` type makes them different (implicit vs ex-

plicit, constrained size vs no size constraint). Also note that in most cases the "conversion from array" can also be implemented via a `bit_cast`, which is not trivially possible for the "load from range" constructor.

This leads us to reconsider whether we should have a load constructor at all. If we have a "conversion from array" constructor the load we can consider "safe" to use is available to users. And given a facility that produces a `span` of static extent from a given range we don't actually need anything else anymore:

```
myvector<int> data = /*...*/;  // std::vector plus span subscript operator
std::basic_simd v = data[i, std::simd<int>::size];
```

The other important consideration is the relation of loads to gathers. Both fill all `simd` elements from values stored in a contiguous range. The load can be considered a special case of a gather where the index vector is an iota vector (0, 1, 2, 3, ...). Thus, if we adopt the `gather_from` function from P2664, the following two are functionally equivalent (using the P1928R12 and P2664R8 API):

```
std::vector<int> data = /*...*/;
constexpr std::simd<int, 4> iota([](int i) { return i; });

auto v1 = std::simd<int, 4>(data.begin());
auto v2 = std::gather_from(data, iota);
assert(all_of(v1 == v2));
```

A high-quality implementation can even recognize the index argument (constant propagated) and implement the above `gather_from` as a vector load instruction instead of a gather instruction.

In terms of API design we immediately notice the inconsistency. These two functions are closer related than the "conversion from array" constructor and "load from range" constructor. Consequently, either there needs to be a gather constructor or the "load from range" constructor should be a non-member function.

```
auto v1 = std::load_from(data);
auto v2 = std::gather_from(data, iota);
```

But these two still have an important difference, the former provides no facility to choose the `simd` width, whereas the latter infers it from the width of the index `simd`. This could be solved with an op-

tional template argument to the `load_from` function. For consistency, the `gather_from` function should provide the same optional template argument:

```
using V = std::simd<int, 4>;
auto v1 = std::load_from<V>(data);
auto v2 = std::gather_from<V>(data, iota);
```

With this definition of the load non-member function, we retain the same functionality the constructor provides (converting loads, choose `simd` width). If we replace the load constructor with a non-member function, we should at the same time remove the `simd::copy_from` member function. The analogue non-member function `std::store_to(simd, range, flags)` replaces `simd::copy_to`. Stores don't require a template argument, since the `simd` type is already give via the first function argument.

Providing loads and stores as non-member functions enables us to provide them for scalar types as well, enabling more SIMD-generic programming. Consider:

```
template <simd_integral T>
void f(std::vector<int>& data) {
  T v = std::load_from<T>(data);
  v += 1;
  std::store_to(v, data);
}
```

This function `f` can be called as `f<int>` and `f<simd<int>>` if `load_from` and `store_to` are overloaded for scalars. With the P1928R12 API the implementation of `f` requires constexpr-if branches.

For completeness, the load and store functions need to be overloaded with a mask argument preceding the flags argument.

§ 10.1. Design alternative replacing `load_from` by `std::ranges::to`

An alternative to providing `load_from` is to use an existing C++ feature called `std::ranges::to` instead. Here are a couple of examples showing the potential of doing this:

```
auto v0 = array{1, 2, 3, 4} | ranges::to<basic_simd>`;

std::vector<double> floats{2.4, 1.1, 9.7, 6.3, 9.6};
```

```
auto v1 = floats | std::ranges::to<basic_simd<int, 3>();
```

The first example works already using the span-constructor proposed earlier. The second example could also be made to work if another constructor were provided which took `std::from_range_t` as its first parameter. Both examples would also require CTAD to deduce the correct simd template parameters.

While `std::ranges::to` does seem to be a potential replacement for `load_from` which is consistent with existing C++ facilities, there are a number of API mis-matches which potentially make this confusing:

- The range will be silently truncated if the output `basic_simd` object is too small. This alone is sufficiently undesirable behaviour to persuade us not to use `std::ranges::to`.
- Other standard types which serve as the destination of `std::ranges::to` can resize at runtime to store all the available input elements. Standard types which can't resize (e.g., `std::array`) are not permitted as the destination of `std::ranges::to`, so `basic_simd` arguably shouldn't either.
- The `std::ranges::to` function would not allow a mask to be passed in, so `load_from` would still be needed for the masked variation.
- There is no obvious way for `std::ranges::to` to replace `store_to`, so `store_to` would still be required.

For all these reasons we do not think that using `std::ranges::to` is a viable full or partial replacement for `load_from`.

§ 10.2. Design alternative combining load and gather / store and scatter

As noted above, an implementation could recognize a constant propagated iota index vector passed to a gather and emit an much faster load instruction. What if we provide a special type denoting a contiguous index range? In that case there could be a single `simd_load(range, simd_or_index_range, flags)` function implementing gather and load efficiently. Example:

```
std::index_range<4> i4 = 1;
std::simd<int, 4> iota([](int i) { return i + 1; });

auto v1 = std::simd_load(data, i4);
auto v2 = std::simd_load(data, iota);
assert(all_of(v1 == v2));
```

Such an `index_range<size>` type is also interesting for subscripting into contiguous ranges, producing a `span` of static extent, which is consequently convertible to `simd`:

```
std::basic_simd v3 = data[i4];
assert(all_of(v1 == v3));
```

§ 11. Data parallel execution policy

Many of the arguments in this proposal concern how explicit the programmer needs to be in what they are doing (e.g., exception or default-initialization) to achieve safe but high-performance. For example, a very common use case for `simd` is to iterate through a large data set in small simd-sized blocks, processing each block in turn. The code might look something like:

```
void process(Range&& r)
{
  constexpr int N = simd<float>::size;

  // Whole blocks
  for (int i=0; i<r.size(); i+=N)
  {
    std::span<float, simd<float>::size> block(r.begin() + i, N);
    doSomething(block); // Standard load - no runtime checks, but its with
  }

  // Remainder
  if (remainder)
  {
    // range checked
    simd<float> remainder(r.begin(), N, simd_flag_partial_loadstore);
    doSomething(block);
  }
}
```

This is efficient and correct, but is verbose common code for the user to write. If an execution policy for `simd` was available then this could be written much more simply, with all the mechanics hidden away.

```
std::for_each(std::execution::simd, std::begin(ptr), std::end(ptr), [&](si
{
    doSomething(block);
});
```

In this case the `simd` library need not worry too much about choosing what default to use in the absence of any flags (e.g., `simd_flag_partial_loadstore`, `simd_unchecked`) because the library code wil use whatever it needs explicitly.

It is not our intent to define an execution policy in this paper, but we point out its potential as it may help guide us to a decision on what `simd` should do by default.

§ 12. Wording

The wording relies on [P1928R12] being landed to the Working Draft. Below, substitute the **?** character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 12.1. Modify [**simd.flags**]

Add the following:

```
// [simd.flags], Load and store flags
?    struct partial-loadstore-flag; // exposition only

?    inline constexpr simd_flags simd_flag_partial_loadstore{};
```

```
// Class template simd_flags overview
?    Every type in Flags is one of convert-flag, aligned-flag, partial-lo
      or overaligned-flag.
```

§ 12.2. Modify [**simd.synopsis**]

Add functions to load a simd value from a range.

[simd.range_load], basic_simd load from range

```
template<class V, class Range, class... Flags>
constexpr V load_from(const Range& r, simd_flags<Flags...> f = {});
template<class It, class... Flags>
constexpr V load_from(const Range& r,
                      const typename V::mask_type& mask,
                      simd_flags<Flags...> f = {});

template<class V, class It, class... Flags>
constexpr V load_from(It begin, std::size_t count, simd_flags<Flags...> f
template<class V, class It, class... Flags>
constexpr V load_from(It begin, std::size_t count,
                      const typename V::mask_type& mask,
                      simd_flags<Flags...> f = {});

template<class V, class It, class... Flags>
constexpr V load_from(It begin, It end, simd_flags<Flags...> f = {});
template<class V, class It, class... Flags>
constexpr V load_from(It begin, It end,
                      const typename V::mask_type& mask,
                      simd_flags<Flags...> f = {});
```

Add functions to load a simd value from a range.

[simd.range_store], basic_simd store into a range

```
template<class T, class Abi, class Range, class... Flags>
constexpr void store_to(const basic_simd<T, Abi>& v, Range&& r,
                        simd_flags<Flags...> f = {});
template<class T, class Abi, class Range, class... Flags>
constexpr void store_to(const basic_simd<T, Abi>& v, Range&& r,
                        const typename basic_simd<T, Abi>::mask_type& mas
                        simd_flags<Flags...> f = {});

template<class T, class Abi, class Out, class... Flags>
constexpr void store_to(const basic_simd<T, Abi>& v,
                        Out begin, std::size_t count,
                        simd_flags<Flags...> f = {});
template<class T, class Abi, class Out, class... Flags>
constexpr void store_to(const basic_simd<T, Abi>& v,
                        Out begin, std::size_t count,
                        typename basic_simd<T, Abi>::const mask_type& mas
                        simd_flags<Flags...> f = {});

template<class T, class Abi, class Out, class... Flags>
constexpr void store_to(const basic_simd<T, Abi>& v,
                        Out begin, Out end, simd_flags<Flags...> f = {});
template<class T, class Abi, class Out, class... Flags>
constexpr void store_to(const basic_simd<T, Abi>& v,
                        Out begin, Out end,
                        const typename basic_simd<T, Abi>::mask_type& mas
                        simd_flags<Flags...> f = {});
```

§ 12.3. Modify [**simd.overview**]

Replace iterator-based constructors with static-span constructors:

> **[simd.ctor], basic_simd constructors**

```
template<class It, class... Flags>
constexpr explicit basic_simd(It first, simd_flags<Flags...> = {});
template<class It, class... Flags>
constexpr explicit basic_simd(It first, const mask_type& mask, simd_fl
```

```
template<class Span, class... Flags>
constexpr explicit basic_simd(const Span& first, simd_flags<Flags...>
template<class Span, class... Flags>
constexpr explicit basic_simd(const Span& first, const mask_type& mask
                              simd_flags<Flags...> = {});
```

Remove `copy_to` and `copy_from` functions.

> **[simd.copy], basic_simd copy functions**

```
template<class It, class... Flags>
constexpr void copy_from(It first, simd_flags<Flags...> f = {});
template<class It, class... Flags>
constexpr void copy_from(It first, const mask_type& mask, simd_flags<F
template<class Out, class... Flags>
constexpr void copy_to(Out first, simd_flags<Flags...> f = {}) const;
template<class Out, class... Flags>
constexpr void copy_to(Out first, const mask_type& mask, simd_flags<Fl
```

§ 12.4. Modify [**simd.mask.overview**]

Remove `simd_mask` memory interaction.

> **[simd.mask.ctor], `basic_simd_mask` constructors**
>
> ```
> template<class It, class... Flags>
> constexpr basic_simd_mask(It first, simd_flags<Flags...> = {});
> template<class It, class... Flags>
> constexpr basic_simd_mask(It first, const basic_simd_mask& mask, simd_
> ```

Remove `simd_mask` `copy_to` and `copy_from` functions.

> **[simd.mask.copy], `basic_simd_mask` copy functions**
>
> ```
> template<class It, class... Flags>
> constexpr void copy_from(It first, simd_flags<Flags...> = {});
> template<class It, class... Flags>
> constexpr void copy_from(It first, const basic_simd_mask& mask, simd_f
> template<class Out, class... Flags>
> constexpr void copy_to(Out first, simd_flags<Flags...> = {}) const;
> template<class Out, class... Flags>
> constexpr void copy_to(Out first, const basic_simd_mask& mask,
>                        simd_flags< = {}) const;
> ```

§ 12.5. Modify [**simd.ctor**] `basic_simd` constructors

Remove iterator-based constructors.

```
template<class It, class... Flags>
constexpr explicit basic_simd(It first, simd_flags = {});

  [remove all descriptions too]

template<class It, class... Flags>
constexpr explicit basic_simd(It first, const mask_type& mask, simd_flags

  [remove all descriptions too]
```

Add static-extent span constructors.

🔗 `basic_simd` constructors [simd.ctor]

```
// Construct a basic_simd from the values contained in the given span-li
// the same static extent.

template<class StaticSpan, class... Flags>
constexpr explicit basic_simd(const StaticSpan& s, simd_flags<Flags...> =

template<class StaticSpan, class... Flags>
constexpr explicit basic_simd(const StaticSpan& s, const mask_type& mask,
                              simd_flags<Flags...> = {});
```

Constraints:

- `element-type<StaticSpan>` must be a vectorizable type.
- The static extent of the `StaticSpan` must be the same as `size`.

Mandates:

- If the template parameter pack `Flags` does not contain `convert-flag`, then the conversion from `element-type<StaticSpan>` to `value_type` is value-preserving.

Preconditions:

- If the template parameter pack `Flags` contains `aligned-flag`, then `to_address(std::begin(s))` points to storage aligned by `simd_alignment_v<basic_simd, span-value-type<StaticSpan>>`.
- If the template parameter pack `Flags` contains `overaligned-flag<N>`, then `to_address(std::begin(s))` points to storage aligned by `N`.

Effects:

- For the unmasked constructor, initializes the `i'th` element with `static_cast<T>(to_address(std::begin(s))[i])` for all `i` in the range of `[0, size())`.
- For the masked constructor, initializes the `i'th` element with `mask[i] ? static_cast<T>(to_address(std::begin(s))[i]) : T()` for all `i` in the range of `[0, size())`.

§ 12.6. Add [**simd.from_range**] `basic_simd` load from range

Add free-functions to load a basic_simd's elements from a range.

`basic_simd` load from range [simd.from_range]

```
(1) template<class V, class Range, class... Flags>
    constexpr V load_from(const Range& r, simd_flags<Flags...> f = {});
(2) template<class It, class... Flags>
    constexpr V load_from(const Range& r,
                          const typename V::mask_type& mask,
                          simd_flags<Flags...> f = {});

(3) template<class V, class Iter, class... Flags>
    constexpr V load_from(Iter first, std::size_t count, simd_flags<Flags
(4) template<class It, class... Flags>
    constexpr V load_from(Iter first, std::size_t count,
                          const typename V::mask_type& mask,
                          simd_flags<Flags...> f = {});

(5) template<class V, class Iter, class... Flags>
    constexpr V load_from(Iter first, Iter last, simd_flags<Flags...> f =
(6) template<class It, class... Flags>
    constexpr V load_from(Iter first, Iter last,
                          const typename V::mask_type& mask,
                          simd_flags<Flags...> f = {});
```

Constraints:

- `std::ranges::range_value_t<Range>` or `iter_value_t<Iter>` is a vectorizable type.
- `Range` models `std::contiguous_range`
- `Range` models `std::sized_range`
- `Iter` models `std::contiguous_iterator`
- `V` is a `basic_simd`

Mandates:

- If the template parameter pack `Flags` does not contain `convert-flag`, then the conversion from `std::ranges::range_value_t<Range>` or `std::iter_value_t<Iter>` to `V::value_type` is value-preserving.

Preconditions:

- If the template parameter pack `Flags` does not contain `simd_flag_partial_loadstore` then the source must be a valid range of equal or greater extent to `V::size`.

- For 3 and 4, `[first, first + count)` must be a valid range.
- For 5 and 6, `[first, last)` must be a valid range.
- If the template parameter pack `Flags` contains `aligned-flag`, then the source range (e.g., `to_address(std::begin(r))`, `to_address(first)`) points to storage aligned by `simd_alignment_v<basic_simd, std::ranges::range_value_t<Range>>`.
- If the template parameter pack `Flags` contains `overaligned-flag<N>`, then `to_address(std::begin(r))` points to storage aligned by `N`.

Returns:

A `basic_simd` object of type `V`. When `Flags` contains `simd_flag_partial_loadstore`, the i'th element of the return value will be:

- For unmasked function: `i < size ? static_cast<typename V::value_type> (to_address(std::begin(s))[i]) ? typename V::value_type{}`.
- For masked function: `(i < size && mask[i]) ? static_cast<typename V::value_type>(to_address(std::begin(s))[i]) ? typename V::value_type{}`.

When `Flags` does not contain `simd_flag_partial_loadstore`, the i'th element of the return value will be:

- For unmasked function: `static_cast<typename V::value_type> (to_address(std::begin(s))[i])`.
- For masked function: `mask[i] ? static_cast<typename V::value_type> (to_address(std::begin(s))[i]) ? typename V::value_type{}`.

§ 12.7. Add [**simd.overview**]

Add class template argument deduction to overview.

[simd.overview]

```
// SIMD ctad.
template <contiguous-range-with-static-extent Rg>
basic_simd(Rg&&)
  -> basic_simd<ranges::range_value_t<Rg>,
                deduce-t<ranges::range_value_t<Rg>, static-range-size<Rg>

template <contiguous-range-with-static-extent Rg, typename... Flags>
basic_simd(Rg&&, simd_flags)
  -> basic_simd<ranges::range_value_t<Rg>,
                deduce-t<ranges::range_value_t<Rg>, static-range-size<Rg>
```

§ 12.8. Add [**simd.ctad**] `basic_simd` class template argument deduction

`basic_simd` class template argument deduction [simd.ctad]

```
template <contiguous-range-with-static-extent Rg>
basic_simd(Rg&&)
  -> basic_simd<ranges::range_value_t<Rg>,
                deduce-t<ranges::range_value_t<Rg>, static-range-size<Rg>

template <contiguous-range-with-static-extent Rg, typename... Flags>
basic_simd(Rg&&, simd_flags)
  -> basic_simd<ranges::range_value_t<Rg>,
                deduce-t<ranges::range_value_t<Rg>, static-range-size<Rg>
```

Constraints:

- `Rg` represents something with a compile-time constant extent which is not larger than an implementation defined maximum.
- `Rg` contains elements of a vectorizable type.

Returns:

- A `basic_simd` parameterized to have exactly the same number of elements as the statically bounded span-like type, and the same element type.

Remarks:

- The span-like object can be a span with known compile-time, or something equivalent (e.g., a C-array with known bounds).

-NOTE: Add suitable note about erroneous behaviour.

§ 13. Polls

§ 13.1. St Louis 2024

Poll: We agree with the direction of P3299R0 regarding non-member functions for loads and stores, and a conversion operator (whether explicit or otherwise) for ranges with static extents.

SF	F	N	A	SA
6	10	1	0	0

§ References

§ Informative References

[P2664R6]

Daniel Towner, Ruslan Arutyunyan. *Proposal to extend std::simd with permutation API*. 16 January 2024. URL: https://wg21.link/p2664r6

[P2876R1]

Daniel Towner, Matthias Kretz. *Proposal to extend std::simd with more constructors and accessors*. 22 May 2024. URL: https://wg21.link/p2876r1

[P3024R0]

David Sankel, Jeff Garland, Matthias Kretz, Ruslan Arutyunyan. *Interface Directions for std::simd*. 20231130. URL: https://wg21.link/p3024r0

[P3116R0]

Zach Laine. *Policy for explicit*. 8 February 2024. URL: https://wg21.link/p3116r0